2

Chapter 2 – Open source business models

You've probably heard the saying "There's no such thing as a free lunch". Do you think this applies to open source? Are the users of open source software getting a free lunch? The answer to this question lies in the famous quote "*free as in free speech, not as in free beer*" discussed in *Chapter 1*. If the creators of open source software are giving their users a free lunch, then how do they themselves earn a living or even be profitable? In this chapter we try to understand the answers to this question.

In this chapter you will learn about:

- The different business models employed by companies (not communities) involved in open source.
- How the business models affect you, as a contributor to open source, if any.

You might be asking yourself "Why should this matter to me as developer?" Strictly speaking, at this point you can choose not to bother; however, from our experience, this knowledge will help you understand some of the nuances behind why communities and companies do what they do. You may use this knowledge to your advantage, and perhaps use it as a guide towards your success when you start your own open source project! Understanding the economics behind how open source operates can be interesting in itself!

2.1 Open source business models: The big picture

An open source business model is a model used by companies that are involved in the development of open source software to keep themselves financially viable and successful. In fact today these companies compete with traditional proprietary software companies for investor's money on the stock markets. Traditional software companies get revenue by the sale of the software they create, that is, they earn money for each copy of the software sold. As illustrated in *Figure 2.1,* traditional software business models monetize software by

either directly selling software products or by providing software development services.

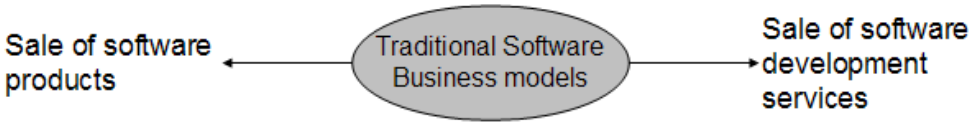


Figure 2.1- Business models for Proprietary software

Did you know?

When IBM sold the first mainframes, the software that was bundled with them was made freely available in open source. Users were allowed to modify and enhance it.

How about open source software companies? Stepping back into the evolution of open source software, it is fair enough to say that the initial roots of open source software were sowed by either community projects which had mutual sharing as their main concern (over business ambitions) as in the case of the GNU project, or funded by government contracts as in the case of BSD UNIX. However as open source software usage progressed to the extent that many of them were viable alternatives to their commercial counterparts, commercial software companies became interested in finding ways by which they could promote, develop and monetize open source software. During this period many startup companies emerged, such as Red Hat. Red Hat focuses on the development and promotion of the Linux open source software, building their revenue and profits around it. These companies began to explore new economic models, different from traditional commercial software to succeed in the competitive software market.

This phase – companies involved in the development and promotion of open source software – has lasted for around a decade or more now. Today there are probably very few domains of product software from operating systems to Business Intelligence, in which there are no commercial companies promoting open source communities and their software. Studies have been carried out by groups to find out the various economic models employed by these companies. The top four models are illustrated in *Figure 2.2.*

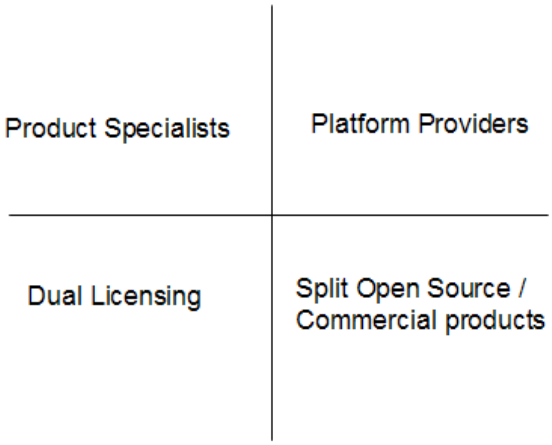


Figure 2.2- Business models for open source software

Companies that develop and promote open source software are sometimes referred to as ***Commercial Open Source Software companies (COSS)*** or ***Professional Open Source Software companies (POSS)***.

The following sections describe in detail each of the four business models shown in *Figure 2.2*.

2.2 Dual licensing

In this model, the open source software is licensed by the POSS company under both, an open source license (using GPL only, a license to discuss in detail in *Chapter 3*) as well as a commercial license. In this model the POSS company generates revenue when it sells the open source software under a commercial license.

Why would a consumer of open source software pay the POSS company to obtain the same software which is also available free of charge? This is needed when the consumer wants to link his own proprietary software to the open source software, but does not want this to cause its proprietary software to become open source as it would under the GPL license. According to the GPL license, when one accepts GPL licensed source code and links it with any other code (dynamic linking or static linking), the linked software also becomes open source. Thus the only way proprietary software vendors can link with GPL software without causing their own software to become GPL is by paying the POSS company. The POSS company then gives the same software to the proprietary software company under a license that excludes the need for the latter to make their proprietary software open source when they link to it. Figure 2.3 summarizes how dual licensing business model works.

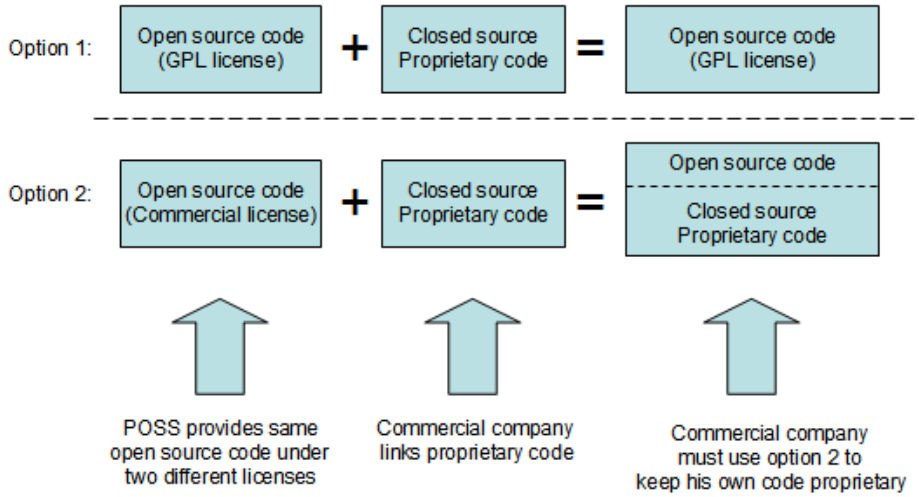


Figure 2.3 - Dual licensing business model

As a developer, you may want to consider this if you are planning to make code contributions to an open source project that has dual license. You will be giving the POSS company the right to make money directly out of your contribution by commercially licensing your code. Of course if the POSS company plans to accept your contributions they will ask you to sign a legal document which states that you have given them the rights to do so.

MySQL, JBoss and SugarCRM are examples of POSS companies that employ the dual licensing business model.

2.3 Split open source software / commercial products

In this model, there is a core portion of the software which is available as open source. This core provides the base functionality. Then there are other portions or extensions that are built on top of or extend the functionality provided by the core. The latter is licensed under commercial or proprietary licenses and sold like typical proprietary software. *Figure 2.4* illustrates how this model works.

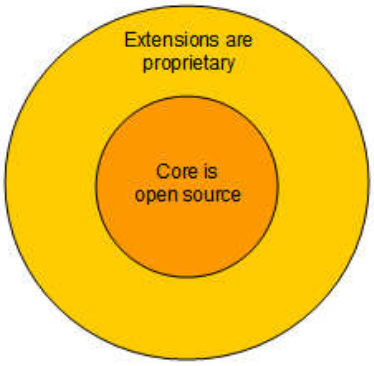


Figure 2.4 - Split open source software / commercial products business model

Many POSS companies choose the Apache or Mozilla open source license when they want to follow this business model, since these licenses allow this kind of intermixing where some parts can be open source and other parts proprietary.

This model is used by many commercial software companies that participate in open source including IBM. In fact, IBM is an acknowledged leader in this space. Does this mean IBM is a POSS company? A company is considered a POSS when most of its revenue comes from open source software sales. Since this is not the case for IBM it would not be considered a POSS company by that definition. Probably the best example of IBM's successful usage of this open source business model is Eclipse – the world's default IDE and tooling platform.

As discussed in *Chapter 1*, before IBM donated Eclipse to the open source community, a number of proprietary Java IDE's competed with each other; yet as a whole, Java IDE's had a very small portion of the IDE market share compared to Microsoft's Visual Studio. When IBM open sourced its VisualAge family of IDE products as Eclipse, it became the de-facto standard and was widely accepted by all. This led to a massive increase in the overall Java IDE market share compared to Visual Studio. Thus, IBM chose the right strategy with Eclipse, and is probably the biggest beneficiary financially - IBM Rational's development tools which are commercial software products have seen a great uptake since they are based on Eclipse.

If you are a developer planning to contribute to an open source project under this business model, you can be certain that your code will never be licensed under a commercial license, since it would be part of the core, which must be open source.

2.4 Product specialists

A POSS company that nurtured an open source software project either by creating it from the start or contributing and maintaining it, can generate revenue from it by providing training and consulting services to the customers of the open source software. This is illustrated in *Figure 2.5*.

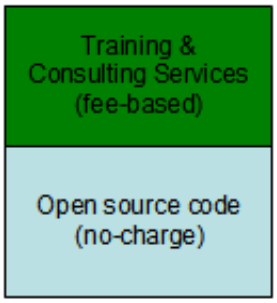


Figure 2.5 - The product specialists' business model

If the open source software caters to a domain that is very complex, and it achieves good adoption, there can be significant revenue to be made from providing training and consulting services. For example, this may be the case with business intelligence software (as compared to wiki software). By the very nature of being the creators and maintainers of the software, it follows that they would be the experts, and hence the best trainers and consultants in the market. The POSS can achieve this position with minimal spending on marketing efforts.

2.5 Platform providers

In a not so distant past software systems were monolithic. In those days one probably bought the entire system from a single vendor who provided all the software and support needed. However these are the days of concepts like Service Oriented Architecture (SOA) where software systems are built from multiple components from different software vendors and integrated into one system. And what if those multiple components were all open source?

Open source software has permeated every domain of product software. Today many software systems in production run entire software stacks consisting of open source software. However, as you can imagine, customers building this type of systems have to face many challenging decisions: Which open source software receives community support? Which ones have a diminishing number of contributors and is on the way out?. How can you determine if a given open source software will work well with another one? How can these software be integrated taking into account different versions? How many resources should be allocated to test and verify the integrated system and then upgrade and patch it in the right manner so that it still works?.

Finding the right answer to these questions can be a nightmare. For example let's say that a patch of open source software X needs Java 1.5; however, open source software Y, which is also part of the system will work only with Java 1.4.

By now, you probably understand why a customer would be willing to pay a company to deliver a tested and verified system made up of different open source software. This is the reason the platform providers business model came into existence. *Figure 2.6* illustrates this idea.

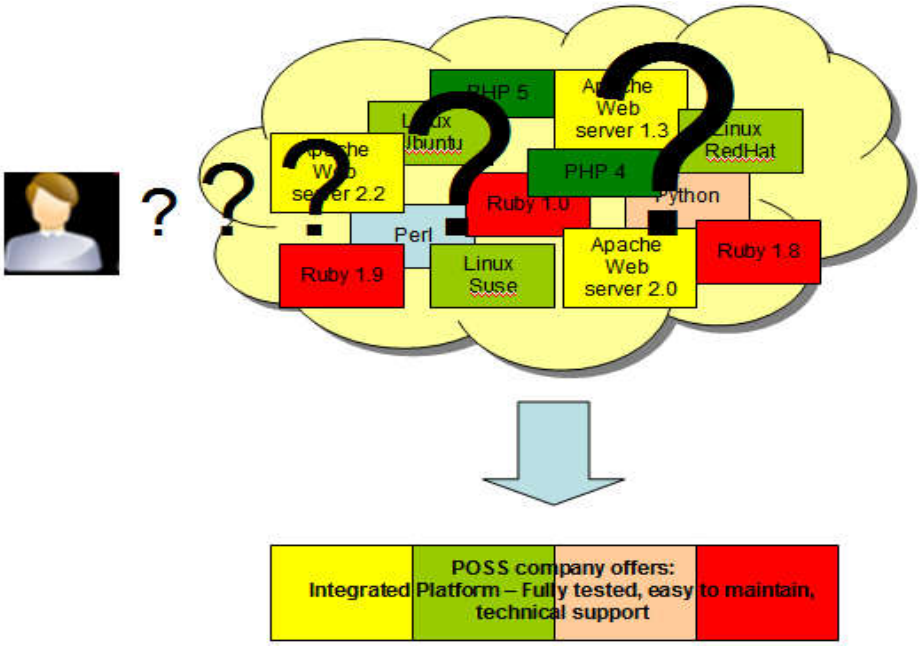


Figure 2.6 - The platform providers' business model

An example of a POSS company using this business model is Zend, "the PHP company". Zend provides a platform fully tested to develop PHP applications.

2.6 Business model relationship to license

It is interesting that when we analyze the above mentioned business models, the relationship these models have is vis-à-vis to open source licenses. This is illustrated in *Figure 2.7*. In the bottom quadrants, the business model is tied to the license of the open source software. Dual licensing applies only when the open source software is GPL-licensed, and Split Open Source / Commercial products when the open source software is Mozilla-based. However the business models in the top quadrant have no bearing on the license of the open source software itself.

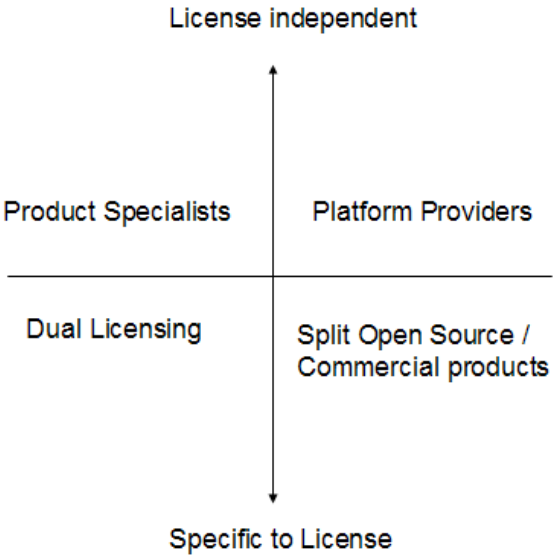


Figure 2.7 - Business model relationship to open source software license

2.7 Open source business model and proprietary software

Customers, startup companies and IT departments are finding it difficult to justify the licensing costs of proprietary software they use, and have been looking at open source software as a cost-reduction alternative. This has taken open source software adoption to new levels especially in segments like Web applications.

In order to remain competitive, several proprietary software vendors have evolved their strategies in this area. A widely used strategy is to offer no-charge versions of their fee-based products. The source code of these no-charge versions remains close. A typical example is with Database Management Systems (DBMS). Established DBMS vendors such as IBM, Oracle® and Microsoft compete today with open source DBMSs like MySQL and PostgreSQL by offering "Express" editions of their popular fee-based DBMS systems. The Express editions are free of charge, and by default have no customer support. This means that if you run into a problem such as a defect in the product, the vendor will not provide you with any support. However, vendors often optionally offer support for a fee. Thus proprietary software vendors are using the same approach as the open source business model – not charging for the software itself but generating revenue by providing support.

In addition to customer support revenue, proprietary vendors view this as an opportunity for adoption and upgrade to fee-based versions in the future. For example a startup company in its initial stages may not be in a position to spend lots of money to purchase software licenses; however, if the company grows, they would need more sophisticated product features which are generally not available in the free versions or on open source software. Therefore, an obvious choice for these companies would be to upgrade to a fee-based version of the proprietary software.

To an extent, Express editions have not seen significant adoption due to the customer's perception that these editions are nothing more than crippled software with upgrade paths to the fee-based versions. And though this may be true in some cases such as Oracle XE and Microsoft SQL Server® Express which place a 2GB hard limit on the amount of data that can be stored; other companies like IBM with its DB2 Express-C database product, do provide more flexibility.

Did you know?

IBM DB2 Express-C database server does not provide any restriction on the amount of data you can store. It can run on Linux, Windows, and the Mac OS, and can be installed on hardware of any size. Applications developed for DB2 Express-C will work with no modification on other editions of DB2, should you need to upgrade, because DB2 Express-C is built using the same code base as the other DB2 editions.

For more information visit ibm.com/db2/express

Did you know?

When Oracle bought Sun Microsystems in 2009, MySQL became an Oracle product since MySQL had been bought by Sun a few years earlier. At the time of writing, Oracle has not made it clear what are its plans for MySQL, but there are interesting comments here:

http://freedb2.com/2009/09/22/ellison-oracle-does-not-compete-with-mysql-mysql-disagrees/

In conclusion, using an Express version does have its merits when the product offers you low cost optional support, flexibility of use, and an easy upgrade path to more sophisticated editions in the future as your needs grow.

2.8 Summary

This chapter started with an explanation about the open source software ecosystem, and how it generates money to sustain itself. It then explained the different business models that exist in the world of open source, and how choosing a model can impact you as a contributor to open source software communities. Finally, we looked at "Express" versions of proprietary software as an alternative for companies to start with no licensing cost, the option for low cost customer support, and the possibility to easily upgrade to a version with more features.

2.9 Exercises

Review the following article which provides a brilliant analogy about a POSS company and bee-keeping.

http://wiki.pentaho.com/display/BEEKEEPER/The+Beekeeper

2.10 Review questions

1. Name a well-known database company that uses the dual licensing business model.
2. In which business model would the Eclipse project and its various plug-ins fall under?
3. What are Professional Open Source Software Companies?
4. Describe how DB2 Express-C is different from its competitors like Oracle XE or Microsoft SQL Server Express.
5. What is the value proposition from businesses using the Platform Providers' Business Model?
6. Which of the following is not one of the top business models used in open source:

   A. Dual licensing

   B. Platform Providers

   C. OS Providers

   D. All of the above

   E. None of the above
7. Which of the following business models require specific licenses?

   A. Split open source / Commercial products

   B. Platform providers

   C. OS Providers

   D. All of the above

   E. None of the above
8. Which of the following is a characteristic of DB2 Express-C?

   A. DB2 Express-C is a free database server by IBM

   B. DB2 Express-C has no limitation on the database size or the number of users

   C. DB2 Express-C can run on Linux, Windows and the Mac OS

   D. All of the above

   E. None of the above
9. What are the advantages of the express versions of some commercial products like DB2 Express-C?

   A. It's free

   B. It has optional fee-based support when your company grows

   C. It provides an easy upgrade path when your company grows

D. All of the above

E. None of the above

10. A company using the "Product Specialists" business model:

A. Does not spend a lot of money in marketing

B. Uses this model because it delivers complex OSS software in a large market

C. Relies on its expertise to deliver training given that it developed the software

D. All of the above

E. None of the above